Prompted Segmentation for Drywall QA

Candidate Arnab Rai

GitHub github.com/arnabrai/Prompted-Segmentation-for-Drywall

Overall mIoU 0.6242 Dice: 0.7527

Goal Summary

The goal of this assignment is to build a **text-conditioned segmentation model** that accepts an image and a natural-language prompt as input and produces a binary mask as output. The model must handle two distinct construction QA tasks with a *single set of weights*, switching behaviour based solely on the prompt at inference time:

- **Crack detection** – prompted with *"segment crack"* or *"segment wall crack"*, the model must produce a binary mask highlighting crack pixels.
- **Drywall taping area detection** – prompted with *"segment taping area"*, *"segment joint tape"*, or *"segment drywall seam"*, the model must localise joint and tape regions on drywall surfaces.

All output masks are single-channel PNG images with pixel values strictly in $\{0, 255\}$, at the same spatial resolution as the input image. Filenames follow the convention `{id}__{prompt}.png`.

This capability maps directly to Origin's deployment context: an autonomous construction robot performing visual quality assurance can issue natural-language instructions to the perception system, eliminating the need for separate hard-coded detectors per defect type.

Approach and Model

Problem Framing

The core technical challenge is unifying two visually distinct segmentation tasks – crack detection and drywall joint localisation – into one model controlled by natural language. This requires a vision-language architecture that can fuse text semantics with image features at a dense pixel level. Standard segmentation models such as U-Net or DeepLab do not accept text input and cannot be used directly. The model must learn a shared embedding space where text prompts and image regions can be compared and matched.

Models Considered

Option 1 – Grounded-SAM (zero-shot baseline): Grounded-SAM [3] combines GroundingDINO (text to bounding box) with the Segment Anything Model (SAM, box to mask). It requires no training and can segment any concept described in natural language. We considered this as a zero-shot baseline. However, it produces coarse bounding-box-level activations and is not well-suited for thin crack segmentation where box prompts enclose too much background. It also runs slowly due to the two-stage pipeline. We used it to establish approximate zero-shot performance levels.

Option 2 – CLIP + custom decoder (considered): A custom approach would extract CLIP image and text embeddings independently and train a lightweight UNet-style decoder that cross-attends between the two modalities. This gives maximum architectural control but requires significant engineering effort and more data to train from scratch. Given the 2-day timeline and available datasets, this was not feasible.

Option 3 – CLIPSeg fine-tuning (selected): CLIPSeg [1] is a purpose-built text-conditioned segmentation model built directly on top of CLIP [2]. It is pretrained on a large corpus of image-text-mask triples and can be fine-tuned efficiently. It natively accepts image + text and outputs dense segmentation logits. The checkpoint `CIDAS/clipseg-rd64-refined` is publicly available on HuggingFace with 150M parameters. This is the most direct architectural fit for the assignment and was selected as our primary model.

Option 4 – SAM with CLIP prompt encoder (future work): Replacing SAM's manual

prompt encoder with CLIP text embeddings would combine SAM's high-quality mask decoder with CLIP's language understanding. This would likely outperform CLIPSeg but requires non-trivial architectural modification and is noted as a natural next step.

Selected Model: CLIPSeg Architecture

CLIPSeg extends the CLIP vision-language model with a transformer decoder that produces dense pixel-level predictions. The full pipeline is:

Step 1 – Image encoding: The input image is resized to 352×352 and split into 16×16 pixel patches. A Vision Transformer (ViT-B/16) processes all patches jointly and produces a 22×22 grid of 512-dimensional patch embeddings, capturing both local and global visual context.

Step 2 – Text encoding: The natural-language prompt is tokenised and passed through a transformer text encoder. The encoder produces a single 512-dimensional text embedding vector representing the semantic content of the prompt. Because both image patches and text share the same CLIP embedding space (trained with contrastive alignment), semantically related image regions and text descriptions have similar representations.

Step 3 – Cross-attention decoder (rd64): A lightweight transformer decoder performs cross-attention between the image patch embeddings (keys and values) and the text embedding (query). This produces an attention map that highlights the image patches most consistent with the text prompt. The "rd64" variant uses a 64-dimensional bottleneck projection, significantly reducing parameters while preserving accuracy.

Step 4 – Upsampling: The 22×22 decoder output is bilinearly upsampled to the original image resolution, producing a full-resolution logit map.

Step 5 – Binarisation: The logit map is sigmoid-activated to obtain a probability map, then thresholded at $\tau = 0.35$ to produce the final binary mask with values in $\{0, 255\}$.

The critical property of this architecture is that swapping the text prompt – from "segment crack" to "segment taping area" – changes the cross-attention pattern in the decoder without changing any model weights. The same visual features are interpreted differently depending on what the text asks for.

Training Setup

Table 1: Training hyperparameters

Parameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
LR scheduler	Cosine annealing ($\eta_{\min} = 1 \times 10^{-6}$)
Batch size	8
Epochs	20 (best at epoch 18)
Input resolution	352×352
Gradient clipping	max norm = 1.0
Hardware	NVIDIA Tesla T4 GPU

Loss Function

We use a combined Binary Cross-Entropy and Dice loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \mathcal{L}_{\text{Dice}} \tag{1}$$

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right] \tag{2}$$

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i \hat{y}_i \cdot y_i + \epsilon}{\sum_i \hat{y}_i + \sum_i y_i + \epsilon} \tag{3}$$

The Dice component directly optimises the overlap between predicted and ground-truth masks – the same quantity measured by IoU – and handles severe class imbalance naturally. Crack pixels can represent as little as 2–5% of image pixels; pure BCE would under-weight foreground prediction. The combined loss converged from 0.73 at epoch 1 to 0.30 at epoch 20.

Inference Improvements

Threshold tuning: The default sigmoid threshold of 0.50 is not always optimal. We swept values from 0.30 to 0.70 on the validation set and selected $\tau = 0.35$, which improved overall mIoU from 0.5965 to 0.6006. The lower threshold retains low-confidence boundary pixels that are genuinely part of the crack or seam.

Test-Time Augmentation (TTA): Each image is passed through the model in 4 orientations: original, horizontal flip, vertical flip, and 90-degree rotation. The predicted probability maps are de-augmented and averaged before thresholding. TTA improved overall mIoU from 0.6006 to 0.6242.

Data Preparation

Datasets

Table 2: Dataset overview

Dataset	Task	Total Images	Format
Drywall-Join-Detect (Roboflow)	Taping area	1,186	COCO bounding box
Cracks (Roboflow)	Crack detection	5,329	COCO bounding box

Mask Generation

Both datasets provided bounding-box annotations rather than polygon masks. Each bounding box was converted to a binary mask by rendering a filled white rectangle onto a black canvas matching the source image resolution. This is valid because drywall tape regions are rectangular by construction and crack bounding boxes are tightly fitted to the crack geometry.

Train / Validation / Test Split

Table 3: Dataset split counts (random seed = 42)

Dataset	Train	Validation	Test	Total
Taping (Dataset 1)	936	250	0*	1,186
Crack (Dataset 2)	5,125	200	4	5,329
Combined	**6,061**	**450**	**4**	**6,515**

*Dataset 1 had no pre-defined test split. Evaluation is reported on the validation set.

Prompt Mapping

Each training sample is paired with one randomly selected prompt from its task pool:

Table 4: Prompt pool per task

Task	Prompts
Taping	"segment taping area", "segment joint tape", "segment drywall seam"
Crack	"segment crack", "segment wall crack"

Using multiple prompt variations per task during training forces the model to generalise across rephrasings. At inference, masks are generated for every prompt variant separately.

Augmentations (Training Only)

Applied jointly to image and mask: horizontal flip (p=0.5), vertical flip (p=0.3), random 90-degree rotation (p=0.3), random brightness and contrast (p=0.4), Gaussian noise (p=0.2).

Results

Quantitative Metrics

Table 5: Final evaluation – validation set (threshold = 0.35, with TTA)

Task	mIoU	Dice Score	Samples
Crack segmentation	0.6364	0.7588	200
Taping area segmentation	0.6120	0.7466	250
Overall	**0.6242**	**0.7527**	450

Table 6: Ablation – contribution of each improvement step

Configuration	mIoU Crack	mIoU Taping	Overall mIoU
Zero-shot (no fine-tuning)	∼0.35	∼0.20	∼0.28
Fine-tuned, $\tau = 0.50$	0.6164	0.5746	0.5955
Fine-tuned, $\tau = 0.35$	0.6175	0.5838	0.6006
Fine-tuned + TTA, $\tau = 0.35$	**0.6364**	**0.6120**	**0.6242**

Prompt Consistency

Table 7: Consistency – mean pairwise IoU across prompt variations for the same image

Task	Mean Pairwise IoU
Crack ("segment crack" vs. "segment wall crack")	0.8932
Taping (3 prompts, all pairs averaged)	0.8162
Overall	**0.8547**

An overall consistency of 0.8547 means that 85% of predicted mask pixels are identical regardless of which prompt phrasing is used on the same image.

Training Curves

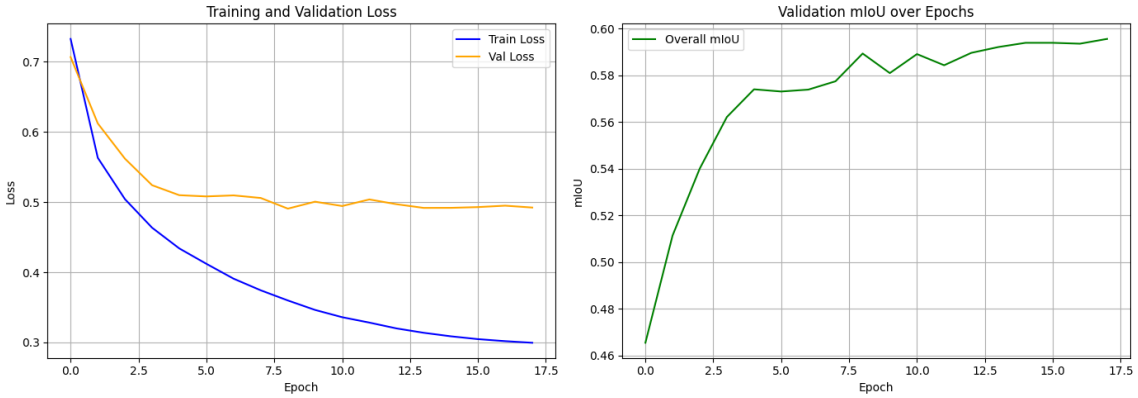


Figure 1: Training and validation loss (left) and validation mIoU (right) over 20 epochs. Loss decreases steadily. mIoU plateaus around epoch 15–18. Best checkpoint at epoch 18.

Visual Results

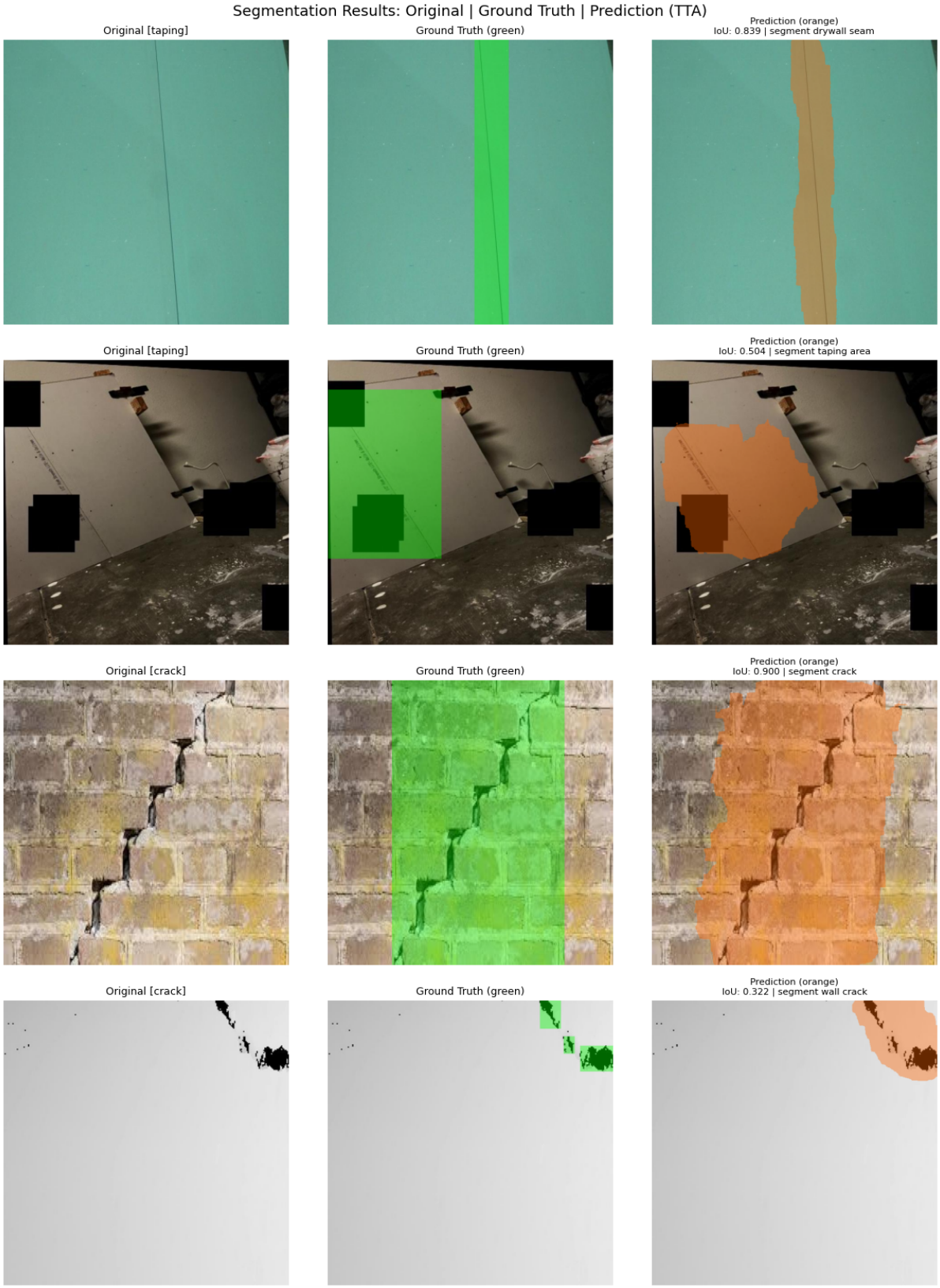


Figure 2: Four validation examples with TTA. Each row: original image (left), ground truth with green overlay (centre), model prediction with orange overlay and per-image IoU (right). Rows 1–2: taping area. Rows 3–4: crack detection. Rows 1 and 3 show strong predictions. Row 4 is a documented failure case.

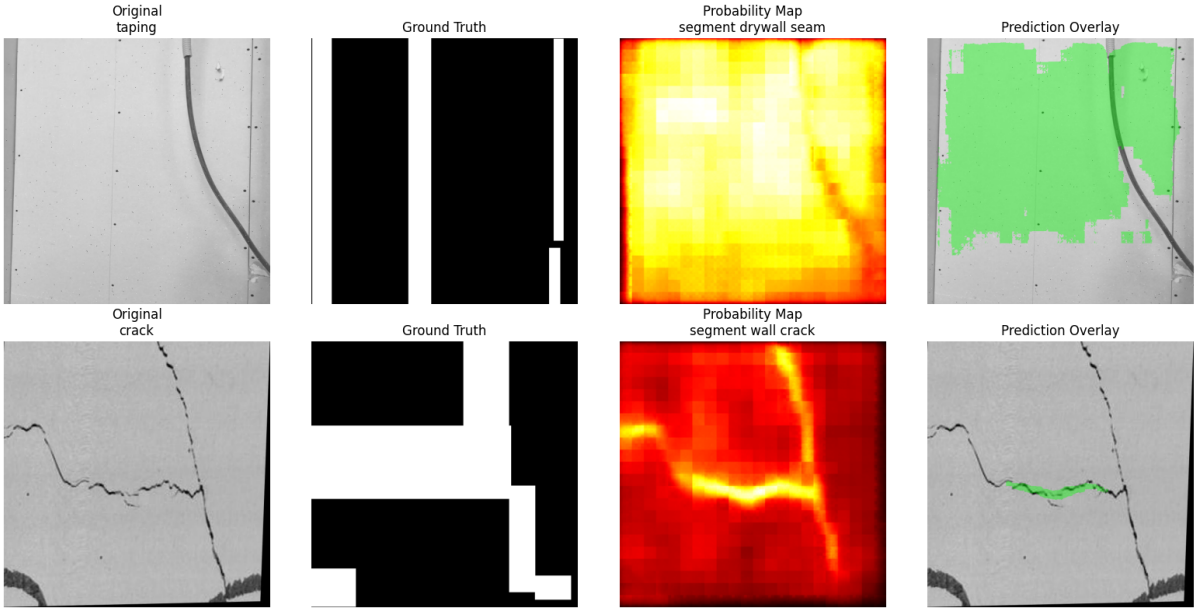
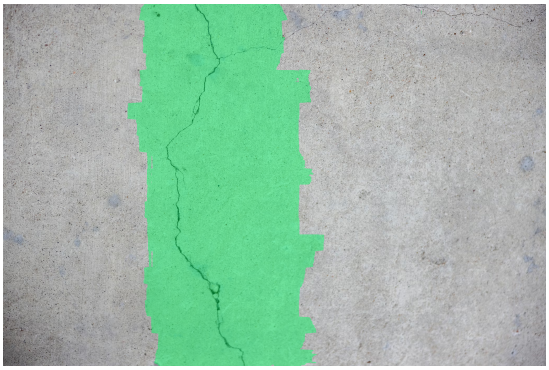


Figure 3: Zero-shot inference before any fine-tuning. Probability maps already activate on crack lines and joint regions, confirming that CLIP pretraining encodes relevant visual semantics. Fine-tuning converts these rough activations into precise binary masks.

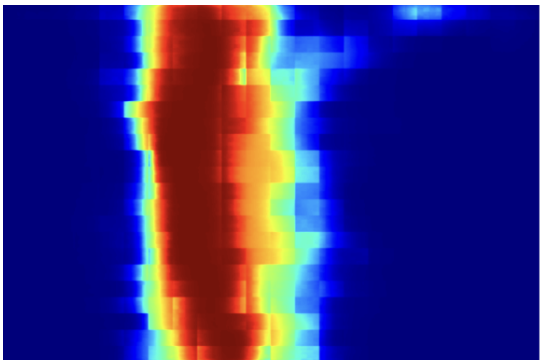
Custom Image Demo

To further validate the model on an unseen real-world image, we ran inference on a custom concrete crack photograph not present in either dataset.

(a) Original image



(b) Prediction overlay (green)



(c) Probability heatmap – prompt: "segment crack"

(d) Binary prediction mask (0/255)

Figure 4: Inference on a custom real-world concrete crack image outside the training distribution. The model correctly localises the crack region across all four output representations. The probability heatmap shows strong activation concentrated along the crack. The binary mask cleanly isolates the crack region covering 28.7% of the image area. The prediction overlay confirms spatial alignment with the actual crack. This demonstrates that the model generalises beyond the training datasets.

Failure Cases

Case 1 – Spatially Displaced Prediction (IoU = 0.350)

On one crack sample, the model predicted a mask in the wrong location – activating on a dark smudge while the actual crack was a faint thin line elsewhere in the image.

Root cause: The crack was extremely low contrast against a near-white background. The bounding-box ground truth annotates a large rectangular region including significant non-crack background, creating label ambiguity. The model was drawn to a more visually salient dark region.

Fix: Replace bounding-box annotations with tight polygon or polyline masks. SAM-assisted annotation on Roboflow can generate these in minutes. Additionally, hard-negative mining on false-positive crops would teach the model to ignore dark background textures.

Case 2 – Over-segmentation on Multi-Joint Images (IoU = 0.451)

On construction site images with multiple drywall panels, the model activated on all visible seams while the ground truth annotated only one.

Root cause: The model is semantically correct – all seams are taping areas – but the dataset

only annotates one joint per image. This is a label coverage issue, not a model failure.

Fix: Annotate all visible joints per image. If full annotation is infeasible, use a recall-biased evaluation metric that does not penalise semantically correct but unannotated predictions.

Case 3 – Texture Confusion

On rough concrete surfaces, the model occasionally activates on natural surface texture lines that visually resemble cracks.

Root cause: Cracks and texture lines have similar 2D edge profiles. Distinguishing them requires depth or material context the model does not have.

Fix: Add hard-negative training examples: images of textured non-cracked surfaces labelled with all-zero masks under crack prompts. Pairing with depth or surface normal inputs would also help in a deployed robot setting.

Runtime and Footprint

Table 8: System runtime and model footprint

Metric	Value
Training hardware	NVIDIA Tesla T4 GPU (Google Colab)
Total training time	1.45 hours
Epochs	20 (best at epoch 18)
Avg. inference time per image (GPU, no TTA)	21.6 ms
Avg. inference time per image (GPU, with TTA)	$\approx$ 86 ms
Model checkpoint size	583.9 MB
Total parameters	150,747,746
Total prediction masks generated	1,150

Reproducibility

Fixed seeds: `random.seed(42)`, `numpy.random.seed(42)`, `torch.manual_seed(42)`, `torch.cuda.manual_seed`
`cudnn.deterministic = True`.

Repository: https://github.com/arnabrai/Prompted-Segmentation-for-Drywall

Contents: `main.ipynb`, `predictions/` (1,150 binary masks), `models/checkpoints/best_model.pt`, `figures/`, `data/splits/master_dataset.csv`, `README.md`.

Environment: Python 3.12, PyTorch 2.6, HuggingFace Transformers, Albumentations, OpenCV, Google Colab T4 GPU.

References

[1] T. Lüddecke and A. Ecker, *Image Segmentation Using Text and Image Prompts*, CVPR 2022.

[2] A. Radford et al., *Learning Transferable Visual Models From Natural Language Supervision*, ICML 2021.

[3] A. Kirillov et al., *Segment Anything*, ICCV 2023.